# Puck Lab: projected workshop runbook

**IIT Kharagpur KDAG × OpenAI Student Collective · 60 minutes planned, room booked for 2 hours as a buffer · about 70 people · 2 hosts with ChatGPT Work · no attendee laptops**

This is the run sheet for a room where only the two hosts have ChatGPT Work and attendees bring no laptops. The student kit still works if you later get participant laptops.

> The room chooses what to investigate. The hosts operate ChatGPT Work. The evidence decides whether the change helped.

## 1. The story in one paragraph

**Problem statement (use this wording on slides):** A planner gets one push to land a puck in a green ring after 35 steps. It picks the push by asking a small neural network to predict how the puck moves. The predicted path ends at or very near the target, but when the real simulator runs the same push, the puck often misses. Work out where the prediction goes wrong, change how the network is trained, and show with evidence whether the change helped.

**Three questions to repeat all session:** What does the model *think* will happen? What *actually* happens? Where do they differ, and why? They match the stages shown on the Lab page (Predict, Execute, Explain).

A small neural network learns how a puck moves on a table with one bumper. A planner uses that network to pick one push that should land the puck in a green target 35 steps later. The network's predicted path looks fine. The real simulator disagrees. Teams work out why, vote on a repair, watch the hosts try it live through ChatGPT Work, stress-test it, and decide from a frozen benchmark whether it helped.

## 2. Outcomes

By the end, participants can:

- Write a ChatGPT Work request with an outcome, context, constraints and a check.
- Explain state, a learned next-step predictor, a rollout, and why small one-step errors can still ruin a plan.
- Choose a hypothesis, say what would disprove it, and update from evidence.
- Judge a change by paired results (what improved, what regressed), not one good-looking demo.

Nobody needs to read the repository. Hosts navigate; teams reason from projected pictures and short explained snippets.

## 3. What is verified, and how

| Fact | Source |
|---|---|
| Original network 1/96, previous improved 73/96, host contact-local model 96/96 on the frozen 96-scene set | Repo results ( `host/results/local-neural-confirmation.json` ); not re-run |
| That 96-scene set has already been scored and shown. It is **not fresh** | `host/local-neural-protocol.json` |
| Student starter: 2/24 development scenes. Adding only `--free-head` : 13/24, 11 new successes, 0 new failures (no hits lost; scene 10's miss grew slightly, 0.130 to 0.133) | Re-run on this machine, 30 Sep 2026; matches `host/results/student-exercise-rehearsal.json` |

| Fact | Source |
| --- | --- |
| Same two checkpoints on the 96-scene set via `host/score_submission.py`: starter **6/96**, free-head **69/96** | Run on this machine, 30 Sep 2026 |
| Contact-weight 5, geometry inputs, and expanded data + geometry each score 0/24 (seed 42, 40 epochs) | Repo rehearsal record; not re-run |
| One `run_experiment.py` run takes about 15 s end to end; `score_submission.py` about 15–18 s | Timed on this machine (CPU). Re-time on the host laptop |
| Contact states are 117 of 4,000 validation states (2.9%) and have about 11× the position error of free-motion states | `host/results/diagnosis.json` |
| Original network, 512 candidates, start (0.16, 0.7): **misses all three presets**. Across 6.8% predicted / 29.1% actual; Diagonal 18.5% / 60.9%; Near side 14.8% / 14.1% (ring radius is 6.5%) | Headless run of `app/engine.mjs` on this machine; Across matches the browser |
| Same three presets with the site's Improved network (512 candidates): actual miss 0.6%, 5.8%, 2.1%, all hits | Same headless run |
| More search does not rescue the original network. At 2,048 candidates actual miss is Across 11.3%, Diagonal 70.9%, Near side 39.0%: still all misses, and two got worse than at 512. Capping push at 0.04 also misses all three | Same headless run |
| Across six exploratory scenes, predicted miss falls as candidates rise (0.204 at 32 to 0.098 at 2,048, cap 0.11) while actual miss stays 0.33–0.45 and hits stay 0–1 of 6 | `host/results/exploratory-scenes.json`; exploratory, three scenes were inspected during development |
| On the 96 scenes, starter baseline vs free-head: mean miss **0.327 vs 0.073**; paired: **63 scenes fixed, 0 lost**, 6 hit by both, 27 missed by both | Run on this machine, 5 Oct 2026 (`score_submission.py`) |
| Across target, 512 candidates: the **starter baseline predicts a 3.2% miss but actually misses by 54.5%**. With free-head weights the prediction and reality **agree (13.6% both)**. At 2,048 candidates free-head hits Across (1.5%) and Near side (2.9%); Diagonal stays a miss (21.6%). Capping the push at 0.04 also hits Across and Near side | Headless run of `app/engine.mjs`, 5 Oct 2026 |
| Two team-style prompts tested in a student-kit copy (sequence training 1/24, mean miss 0.308; geometry inputs 0/24, mean miss 0.410), about 3 minutes for both | Rehearsal by ChatGPT Work on the host laptop, 6 Oct 2026; I did not re-run them. The geometry result matches the repo record. |
| No-geometry control: a plain network (no routing, no surface coordinates) trained on the large expanded dataset scored **2/96** (40 passes, mean miss 0.476) and **6/96** (200 passes, width 128, mean miss 0.340), against 6/96 for the starter and 69/96 for the free-motion helper. On the 24 development scenes both scored 1/24 | Run on this machine, 6 Oct 2026. One seed; a plain network, not the host model with its routing removed |
| Original training data: 932 of 30,000 examples involve a wall or bumper hit (147 the bumper), speeds 0.005–0.040. Expanded data: 23,336 of 177,319 (13.2%), speeds up to 0.115, 65% above 0.04. A hit means the puck is about to touch a wall or the bumper on the next step | Counted from `assets/training-observations.npz` and `assets/neural-training.npz`, 6 Oct 2026 |

| Fact | Source |
| --- | --- |
| **The full contact-local model rebuilt with no hand-given geometry** (`host/geometry_from_data_pipeline.py`). Step 1 infers the geometry from the original 30,000 transitions: drag 0.985, walls 0.0200 / 0.9800, bumper centre (0.5000, 0.5000), radius 0.1150, matching the true values to four decimals. Steps 2 and 3 train the learned gliding, wall and bumper pieces (collisions in surface coordinates) using only those inferred numbers. Result: **96/96 on the held-out scenes (mean miss 0.0135) and 24/24 on development** with the larger provided dataset, and **94/96 (0.0141) and 24/24** using only the original 30,000 transitions. The free-motion-helper repair using the same inferred geometry scored 13/24 and 69/96, identical to the hand-given version; with a deliberately wrong bumper radius (0.25) it fell to 6/24 | Run on this machine, 6 Oct 2026. Assumes the form of the physics (position moves by velocity, drag, reflecting walls, one round bumper) and infers the numbers; noiseless data; one seed (373), 1,000 epochs and the 3-32-32-4 bumper network are the host's existing settings, not tuned on the 96 scenes here; the model is scored by script and cannot be loaded in the Lab |
| In the browser (host site), the new approach on the Across shot with 512 candidates: predicted miss 1.6%, actual miss 1.6%, "Target reached" (the predicted and actual paths coincide) | Played in the browser on a local copy of `dist-host/`, 6 Oct 2026 |
| 25 JavaScript tests pass (the earlier 19 plus 6 for the new approach: browser/Python parity, recovered geometry, invalid geometry rejected, wrong geometry changes the prediction, known geometry unchanged, Across target hit) | Run on this machine, 6 Oct 2026 |
| The 19 earlier JavaScript tests pass | Run on this machine |
| The site's team-model loader accepts the free-head export | Code inspection (`app/engine.mjs`); confirm on the day |

Not verified: room-scale timing, projected legibility, the Jupyter UI, the manual-push control, `--data expanded` on its own, and results for any change other than those listed. Rehearse these and record what you see. **Do not promise an outcome you have not seen.**

**Playback rule:** shots animate with `requestAnimationFrame`. In testing, a shot started while the page was hidden paused with the controls locked, then finished normally (same result) as soon as the tab was shown again. Keep the demo tab in front while executing; if it looks stuck, bring the tab forward before reloading.

## 4. Room and roles

- **10 teams of about 7**, one printed worksheet each (Section 9). Everyone predicts alone first, then discusses.
- Team roles, rotating: discussion lead, recorder, spokesperson (casts the team vote).
- **Host 1 (operator):** projects the site and ChatGPT Work, runs every command, inspects diffs, manages checkpoints.
- **Host 2 (facilitator):** frames each vote, counts team votes (one raised hand per team), takes two 30-second contrasting reasons, keeps time. Host 2's own ChatGPT Work account and laptop are the backup with the same workspace.
- Run the host site locally with `start.ps1` (http://127.0.0.1:8766/, opens in host view). The student-only site is `start-student.ps1` (http://127.0.0.1:8767/). The hosted copy is owner-only until shared.
- **Two views.** The default is the **student view**: only the original network and the team model are selectable, and the Results tab and improved-model text are hidden. **Host view** (`?host=1`) unlocks the **known-geometry model**, the **new approach (geometry inferred from the data)**, the physics reference and Results. Project the student view for the whole session, either http://127.0.0.1:8766/app/ or the student site on :8767, and open http://127.0.0.1:8766/app/?host=1 only at the reveal in 6.8. Switching reloads the page and clears the shot log, so screenshot it first. Leave host view with the "Host view · exit" chip.

## 5. Agenda

**The session is planned for 60 minutes and follows the deck ("ACTUAL WS PPT THNX", 19 slides).** The room is booked for 2 hours, so there is a large buffer. Use it only through the stretch blocks below, in order, and only when you are on time and the room is engaged. There is no long break (keep water on the tables). The stages follow the Collective's standard run of show, with one change: **the first stage is check-in only.** Attendees do not need ChatGPT Plus or any account for this session, so there is no access step.

**The core idea of the middle:** the hosts show two worked examples, then **the teams use their own brains: they write their own prompt, hand it in, and the hosts run it live in ChatGPT Work.** Slide 14 stays on screen while the hosts work in the site and the tool.

| Time | Slides | Stage | Segment | Output from the room |
|---|---|---|---|---|
| 0:00–0:05 | 1–2 | Welcome and check-in | Check-in form, title | Everyone checked in and seated |
| 0:05–0:15 | 3–8 | Set the scene | The Collective, campus leads, form teams, ChatGPT Work, the activity and QR, activity title | Teams of 6–7 with roles; site open on phones |
| 0:15–0:21 | 9–10 | Show what's possible | The puzzle, neural nets in plain words, what to try | Written predictions; shared vocabulary |
| 0:21–0:28 | 11–13 | Get ideas flowing | Choose the first idea, starter prompt and one check, group discussion | Hypothesis voted and revised from evidence |
| 0:28–0:36 | 14 | Build together | Two worked examples: baseline with collision weight 5x, then the free-motion helper | Predicted, then measured, results |
| 0:36–0:42 | 14 | Your turn | Teams write their own prompt; spokespeople hand them in | One written prompt per team |
| 0:42–0:52 | 14 | Build together | Hosts run two team prompts live in parallel; one stress case while they run; read results vs the baseline | Team ideas tested, results read honestly |
| 0:52–0:57 | 15 | Spotlight | The real test on 96 scenes for the best candidate, accept / reject / qualify vote, two teams | A fair score; two teams spotlighted |
| 0:57–1:00 | 16–19 | Wrap up | Check-out, stay connected, fun fact | One trusted result, one next experiment |

**Timing rule:** protect the last 8 minutes (the real test, vote, wrap-up). If you are running late, do not extend. Cut in this order:

1. Cut the stress case (6.6).
2. Skip worked example 1 and go from the baseline straight to the free-motion helper.
3. Run one team prompt, not two.
4. Skip the reveal on slide 15.
5. Never cut the freeze-and-score steps of the real test.

If ChatGPT Work stalls, use a clearly labelled prepared result (Section 8).

### If the session extends: stretch blocks

Check the clock at the end of each segment. If you are on time or ahead and the room is engaged, add the next block, in this order. Each block goes inside the segment named, so later start times move by the minutes added.

| Order | Add to | What to add | Minutes | Session ends at |
|---|---|---|---|---|
| 1 | Diagnose (6.4, slide 12) | Run the other two checks: one-step vs rollout error, and the optimism scatter | +3 | 1:03 |
| 2 | Your turn (6.5b, slide 14) | A third team prompt run live, or a second round of prompts | +5 | 1:08 |
| 3 | Break it (6.6, slide 14) | More cases: limit the push, predict-same-push, and the manual push | +5 | 1:13 |
| 4 | The puzzle (6.2, slide 9) | The other two presets (Diagonal, Near side) | +3 | 1:16 |
| 5 | Net essentials (6.3, slides 9–10) | A fuller train, validation and test explanation, and why choosing on the final set is unfair | +3 | 1:19 |
| 6 | Benchmark (6.7, slide 15) | Walk through one scene the repair fixed and one it still misses | +4 | 1:23 |
| 7 | Review (6.8, slide 15) | A longer team debate and questions before the reveal | +5 | 1:28 |

With every block added, the session ends at about 1:28 and keeps about 32 minutes before the 2-hour booking ends. Aim to leave the room at least 10 minutes before the booking ends.

## 6. Segment guide

### 6.1 Set the scene (0:05–0:15, slides 3–8)

1. **The Collective and the campus leads (slides 3–4, 2 min).** What the OpenAI Student Collective is, then each lead's intro and fun use-case.
2. **Form teams (slide 5, 3 min).** Groups of 6–7 (the slide says "no pun intended"), so about 10 teams and one vote per team. Introductions: name, major, year, hometown, and "something you've been unusually into lately". Roles: discussion lead, recorder, spokesperson. Hand out the worksheets.
3. **ChatGPT Work, the activity and the QR (slides 6–8, 5 min).** What ChatGPT Work is (take on whole projects, connect to your tools, run proactively), read the activity line, have the room scan the QR to open the site, then the activity title.

The read-only starter prompt is run later, on slide 12. Say aloud: ChatGPT Work helps us change the program. The puck model is a separate, small network running locally.

### 6.2 The puzzle (0:15–0:19, slide 9)

On the site, choose **Original neural network**, 512 candidates, wide range. Pick a target preset. Before each execute, teams write on the worksheet: will the puck land in the ring? Then **Find a new shot**, show the dashed predicted path, ask the room, then **Execute**.

Reference shot (seen today): target (0.83, 0.72), original network, 512 candidates. Predicted miss 6.8%, actual miss 29.1%; predicted route 0.64 and actual 0.78 table widths. The prediction looked confident and the simulator disagreed.

The other presets behave the same way for this model: Diagonal predicts 18.5% and actually misses by 60.9%; Near side predicts 14.8% and misses by 14.1%. Use the Across preset only for now. The other two presets are stretch block 4.

Ask the site's prompt: *would trying more candidate shots make the robot more reliable?* Take a vote and hold the answer for the diagnosis.

**Fix slide 9 before the session.** It tells teams to "go to the team kit, install the files given". PyTorch is a large download that will not work on venue Wi-Fi mid-session. Reword it to: follow along on the Team kit tab if you have a laptop (optional). The hosts run everything live.

### 6.3 Neural-net essentials and the rules (0:19–0:22, slides 9–10)

One diagram: state `[x, y, vx, vy]` → network → next state, repeated 35 times to make a path. The planner scores 512–2,048 candidate pushes with those predicted paths; the real simulator executes only the chosen one.

Cover training vs validation vs testing in one sentence each. The fuller version is stretch block 5. Ask why choosing a model using the final benchmark would be unfair.

**Freeze now, and write it on the board:**

- Fixed for every comparison: planner, action pool, horizon 35, speed cap 0.11, target radius 0.065, evaluator.
- Development set: 24 scenes (used to choose). Final set: the 96-scene set (used once).
- Success is a landing inside the ring, reported as hits out of the total, alongside mean miss.
- A repair must change the **learned predictions**. Replacing the network with physics equations is a different experiment.

### 6.4 Diagnose (0:21–0:28, slides 11–13)

Slide 11 gives three options for "what will the model get wrong first". Give teams one minute:

1. **Collisions:** the network is wrong at bumper and wall collisions, which are rare in the data.
2. **Long free motion:** small errors accumulate over 35 steps of gliding.
3. **The planner's picks:** the planner chooses shots where the network is most over-optimistic.

Slide 12 follows: run the read-only starter prompt, then the checks. Slide 13 is the group discussion (90 seconds).

Take one vote per team, hear two contrasting reasons (30 seconds each), then run the **budget sweep** and show the **contact-vs-free table** (a stored result, nothing to run). The other two rows are stretch block 1. Do not announce which hypothesis is true first. Each check uses the notebook logic in `student/team-notebook.ipynb` and takes seconds:

| Check | What it shows | Tests |
|---|---|---|
| Budget sweep: 32 / 128 / 512 / 2,048 candidates, same target | Whether more search helps | 3 |
| Contact vs free error table (from `host/results/diagnosis.json`) | Contact states are 2.9% of samples with about 11× the error | 1 |
| One-step error vs free-rollout error over 35 steps | Whether error starts at once, grows, or jumps at contact | 1, 2 |
| Predicted vs actual miss across the candidate pool, planner's pick highlighted | Whether selection favours optimistic errors | 3 |

Prompt for each check (Host 1):

```
Run only this diagnostic: [budget sweep / contact-vs-free error / one-step vs
rollout error / optimism scatter], using the notebook's code and the original
network on the reference target. Do not edit files. Show a compact table or
plot, explain it for a beginner, and say what it does and does not establish.
Report any failure to run.
```

Rule out hypotheses with what appears on screen. A diagnostic supports an explanation; it does not prove a cause. Expect the budget sweep to show option 3 at work: predicted miss keeps falling while the actual miss does not (see Section 3). More search finds shots the network *likes*, not shots that work.

### 6.5 Improve: two worked examples (0:28–0:36, slide 14)

Slide 14 fixes the order. **Improvement 1** runs the baseline, then the same model with collision loss weighted 5x. **Improvement 2** is the guided `--free-head` step. There is no room vote on which repair; the room predicts (all fixed, some, or none) before each run. Allow about four minutes each, including the baseline run. The room's own prompts come next, in 6.5b.

Menu (each is one single-factor change from the starter; all from `student/EXPERIMENTS.md`):

| Option | Command | Rehearsal result (24 dev scenes) | Mean miss (table widths) |
|---|---|---|---|
| Starter baseline | `run_experiment.py --name baseline` | 2/24 | 0.385 |
| Weight collision loss more | `--contact-weight 5` | 0/24 (worse than 2/24) | 0.545 |
| Add geometry inputs | `--geometry` | 0/24 | 0.410 |
| Train on the larger dataset | `--data expanded` | Not measured alone (expanded data with geometry: 0/24) | 0.508 (with geometry) |
| A separate learned head for free motion | `--free-head` | 13/24, no hits lost (one scene's miss grew slightly) | 0.101 |

Baseline and free-head were re-run on this machine; the others are from the repo's rehearsal record (seed 42, 40 epochs). Show **mean miss next to hits** every time: "0.385 to 0.101" reads as progress where "2 to 13 out of 24" can look like a lottery, and it shows that the three failed options made the miss *larger*.

Run the baseline first (2/24) so every result has a paired comparison. Say plainly that plausible repairs can fail; the 0/24 results are useful evidence. Present `--free-head` as a **guided step** that follows the evidence from 6.4 (errors that build up over 35 steps), not as something teams were expected to discover. The 96/96 reveal is a lesson about structure, not a puzzle they failed to solve.

For each repair, before running, ask teams to tick *all fixed / some improve / no clear gain* on the worksheet. Then:

```
From student/, run: python run_experiment.py --name [name] [flags] --compare baseline
Do not change the planner, scenes, seed or evaluator. Show the printed
result, the changed settings, and every new success and new failure. Do not
explain a cause until we discuss it. Report any command that fails.
```

Run names must be new each time (the tool refuses to overwrite). Keep failed runs.

After the run, load its `network.json` through **Try your team's trained model** on the site and execute the reference shot with it, so the room sees the puck move on the real weights.

Explain the free-head change in plain words: it fits a separate small layer to free-motion data and routes non-contact states to it. It uses geometry to decide which branch runs, which is an explicit prior and not a discovery by the network. Whatever you run, one repair is not a proof of cause.

### 6.5b Your turn: team prompts (0:36–0:52, slide 14)

After the two worked examples, the teams use their own brains and the hosts run what the teams decide.

**Write (0:36–0:41, five minutes, hard stop).** Each team writes **two ideas** it would change, then prompts for them. "Pick ONE change per idea. Say what you expect and what would change your mind. Write it as a prompt for ChatGPT Work." Use this recipe (put it on the worksheet and, if you add a slide, on the screen):

- **Goal:** what you want to improve.
- **Change:** one thing to change.
- **Keep fixed:** the planner, the scenes and the seed.
- **Check:** show hits, mean miss, and every new success *and* new failure against the baseline.

Both hosts walk the benches and ask: what do you expect, and what would change your mind?

**Hand in (0:41–0:42).** Each team puts both ideas on one slip. The spokesperson reads only the best one aloud (one sentence); you scan the slips for the second. Pick **two** prompts to run now, one per host, and prefer two different ideas.

**Triage rules.** Run a prompt if it changes one thing, keeps the planner, scenes and seed fixed, changes the *learned* model (training, inputs, loss or architecture, not physics equations), and could finish in a few minutes. Redirect the others kindly: "great idea, not today; we'll note it as a follow-up." Typical redirects: it needs more data than the kit has, it replaces the model with physics, it asks to see the held-out scenes, or it changes the planner.

**Hint ladder (give hints in this order, only as far as a stuck team needs).** Do not give the free-motion helper as a hint; it was the second worked example.

| Level | What you say | Examples |
|---|---|---|
| 1. Questions | Nudge with a question, not an answer. | "Where did the two paths split apart, before or after the bumper?" "What does the model get to see about the walls and the bumper?" "Does it learn one step at a time, or whole stretches?" "How big is the network, and how long did it train?" |
| 2. Directions | Point at a *kind* of change. | Change **what it sees** (extra inputs). Change **how it learns** (train on short sequences instead of single steps). Change **how big or how long** (wider network, more training passes). Change **what data it gets** (more fast pushes, more examples near the bumper). |
| 3. Concrete options | Only if still stuck, name something that runs quickly. | `--width 64`, `--epochs 80`, `--geometry`, `--data expanded`, or a sequence-training edit (about 3 minutes, see the rehearsal below). |

**Advanced idea for a strong team (do not offer it first):** "Don't be handed the geometry. Infer it from the data." Look for the steps where the puck does something the gliding law cannot explain, work out where the walls and the bumper must be, then use those inferred numbers to decide which steps are collisions. Rehearsed: it recovers the geometry exactly (see Section 3) and `host/geometry_from_data_pipeline.py` shows one complete way, from data to a 96/96 model. Note that adding geometry only as extra *inputs* (`--geometry`) scored 0/24; the gain comes from using it to route, as the free-motion helper does.

**Sentence starters for the prompt:** "Goal: reduce the miss on **." "*Change:*** ." "Keep fixed: planner, scenes, seed." "We expect ___ because **. *We would change our mind if*** ."

**What you already know about these ideas (do not say it before the run):** extra wall and bumper inputs ( `--geometry` ) scored 0/24 in rehearsal, and heavier collision weighting scored 0/24. Sequence training gave a mixed result. A wider network, more epochs, and expanded data on its own have **not** been measured, so treat any of those as a genuine unknown and say so.

**Guardrail suffix.** Add this to every team prompt before you paste it:

```
Work only in this folder copy. Keep the planner, scenes, seed and evaluator fixed.
Use a new run name and keep the baseline. Run it on the 24 development scenes
with --compare baseline. Change only what the prompt asks, list the files you
changed, and do not look at or use any held-out scenes. Show hits, mean miss,
and every new success and new failure. Do not claim a cause.
```

**Setup (do this before the event).** Run two chats, one per host, each in its **own copy of the student kit**. Two chats never overwrite each other's runs, and the held-out scenes are not visible to a team prompt. Keep the full `puck-lab` folder for the real test only. Rehearse with two or three invented prompts, for example a wider network ( `--width 64` ), more epochs ( `--epochs 80` ), and a code edit such as a multi-step loss.

**While they run (0:44–0:48).** Run one stress case on the site (6.6) so the room is not watching a spinner. Go back to the chats at 0:48.

**Read the results (0:48–0:52).** For each prompt say: the team, the idea, hits, mean miss, and new successes and new failures against the baseline. Credit the team. A worse result is a good result if you explain it. If a chat is still working at 0:50, say "we'll check back at the end."

**Rehearsal of two team-style prompts (6 Oct 2026, run by ChatGPT Work in a copy of the student kit on the host laptop; I have not re-run these myself).** Both were tested separately against the same baseline, with the data split, seed, planner and scenes fixed. It took **2 min 57 s** for both, including adding a new option to the training code.

| Idea (as a team might write it) | Hits (of 24) | Mean miss | What changed |
|---|---|---|---|
| Baseline | 2 | 0.3845 | |
| Train on short sequences (5 steps) instead of single steps | **1** | **0.3083** | Scene 20 fixed; scenes 6 and 21 lost. Mean 35-step prediction error fell from 0.2617 to 0.1916. |
| Add wall and bumper distances as inputs ( `--geometry` ) | 0 | 0.4103 | Scenes 6 and 21 lost. Collision-position error improved (0.01291 to 0.01131) but target hits got worse. |

How to narrate this: neither idea raised the hit count. **Sequence training is a mixed result** (the average miss improved and the prediction error fell, but a hit was lost). It is a good example of why you show mean miss *and* hits, and why you do not call it a win. **The geometry idea predicted collisions better and still did worse on targets**: better predictions near collisions were not enough on their own. If a team picks one of these, you already know roughly what to expect.

**What is still not measured.** The time for other code-edit ideas, and how parallel chats behave on the venue network. Rehearse with other invented prompts.

**Reset the folder copies before the event.** That rehearsal edited `student/run_experiment.py` and `student/train_repair.py` in the copy it ran in. Make the two live copies fresh from `packages/student-kit.zip` so the chats start from the original files and the rehearsal runs do not show up as extra scenes or results.

### 6.6 Break it (0:44–0:48, while the team prompts run)

Use the site's controls; teams choose, Host 1 operates. Start each case by asking for a prediction. Run it while the team prompts are working, about four minutes: move the target (Diagonal) and, if there is time, change the search effort (2,048 candidates). With the free-head weights loaded, see Section 3 for what to expect. Predict-same-push, limit-the-push and the manual push are stretch block 3.

| Stress case | Control | Ask |
|---|---|---|
| Move the target | Click the table or use Across / Diagonal / Near side | Does the repair keep working away from the reference shot? |
| Limit the push | Wide range (0.110) vs original training range (0.040) | Does staying inside the trained speeds fix it? For the original network at 512 candidates, no: all three presets still miss. |
| Change search effort | Slider from 32 to 2,048 candidates | Does more search fix a wrong predictor? At 2,048 the original network still misses all three, and Diagonal and Near side get worse than at 512. |
| Predict same push | Switch model, then *Predict same push* | Do the models disagree about the same physical push? |
| Manual push | The site's *Can you choose a better push?* control | Can a human beat the planner? Rehearse the control first. |

Log every executed shot in the site's experiment log and keep a screenshot of it. Do not start a new repair after seeing stress results.

Say plainly: these are chosen stress cases, not a robustness benchmark. The simulator is deterministic and noiseless with one bumper.

### 6.7 Benchmark: the real test (0:52–0:55, slide 15)

Slide 14's **Improvement 3 is a team task** that works with the student kit alone: freeze the best run and write the report from its `score.json` (the notebook's last cell writes `team-report.json`): hits, mean miss, new successes *and* new failures vs the baseline, and what it does not show. The 96-scene scoring below is the hosts' "real test" right after. It needs the full `puck-lab` folder.

1. **Freeze (15 s).** Confirm the candidate's `network.json` hash and the code state. No further edits.
2. **Score (1.5 min).** From the puck-lab root, once per checkpoint:

`text python host/score_submission.py path/to/baseline/network.json python host/score_submission.py path/to/candidate/network.json`

Copy each `network.json` to a scratch folder first: the scorer writes its report next to the file it scores. 3. **Interpret (1 min).** A scene-by-scene walkthrough is stretch block 6. Show hits out of 96, the paired new successes and new failures, and one scene the repair fixed and one it still misses. Ask teams whether the result matches their prediction. 4. **Decision (15 s).** State what the numbers do and do not show.

**How to run the real test live (about 3 minutes).**

1. **Before the event:** make a `scratch/` folder inside the full `puck-lab` folder. After the team runs, copy the two `network.json` files you want to compare into it, renamed so the reports are easy to read: the baseline from `student/results/runs/baseline/` in your chat copy as `scratch/baseline.json`, and the candidate (the free-

motion run, or a team's run if it clearly beat the baseline) as `scratch/candidate.json`. The scorer writes its report next to the file it scores, so always score copies.

2. **Freeze (optional, 15 seconds):** show the two files' fingerprints so the room sees nothing changes after this point: `Get-FileHash scratch\baseline.json` and `Get-FileHash scratch\candidate.json` in PowerShell.
3. **Score, once each** (about 15 to 18 seconds per file), from the full `puck-lab` folder:

```
python host/score_submission.py scratch/baseline.json
python host/score_submission.py scratch/candidate.json
```

Each prints `confirmation: N/96 targets.` and writes `scratch/baseline-scored.json` and `scratch/candidate-scored.json`.

4. **Get one table with everything** (hits, mean miss and the paired counts, about 40 seconds for two models). This replaces running the two commands above separately:

```
python host/show_scores.py scratch/baseline.json scratch/candidate.json
```

It prints, for each model, hits and mean miss on the 96 scenes, and for every model after the first, how many scenes it fixed or lost compared with the first (the baseline). Nothing is written next to your files.

5. **If scoring fails or ChatGPT Work stalls:** report what completed, then show the prepared rehearsal numbers (6/96 to 69/96, mean miss 0.327 to 0.073) clearly labelled as rehearsal. Do not present saved numbers as a live result.

On this machine the starter scored 6/96 and the free-head repair 69/96. Treat those as rehearsal values, not what to expect live.

**Honesty rules for this stage:**

- The 96-scene set has been scored and displayed before. Present it as a public benchmark, not a fresh holdout. If you want a fresh set, generate one with a new seed using `scene_set` in `host/controllers.py`, write it into a new protocol file, and freeze it before the event.
- Do not tune after seeing benchmark results.
- One checkpoint, one seed, one deterministic simulator. Say so. Do not say "significant" or "robust".
- If scoring fails, report what completed and stop.

### 6.8 Engineering review and spotlight (0:55–0:57, slide 15)

Team vote: **accept**, **reject**, or **qualify** the repair. Two teams with different views give 30-second reasons citing something they saw on screen. These two teams are your spotlight: invite them to the front.

Then switch to host view (`?host=1`) and show the **Known-geometry model** and **New approach** options and the **Results** tab (1/96, 73/96, then 96/96 for the known-geometry model and 96/96 and 94/96 for the new approach, on the same 96 scenes). Explain that the newest model stores collision responses in surface-relative coordinates and shares its training data with the previous one. Changes were made together, so this does not isolate one cause. The host model has never been run on a fresh set beyond that one. If time is short, skip the reveal.

### 6.9 Wrap up and check-out (0:57–1:00, slides 16–19)

Reinforce these points (the script gives the first three on slide 15 and the closing line on slide 16; use the rest if time allows):

1. A ChatGPT Work request needs an outcome, context, constraints and a check.

2. Low one-step error does not mean a good plan.
3. A plausible repair can fail; keep the negative results.
4. Compare paired results, including regressions.
5. Confidence in a prediction is not evidence it is correct.

Exit question (optional, slide 16): *Name one result you trust, the evidence for it, and one uncertainty you would test next.*

Closing line: "ChatGPT Work can implement the change. Your job is to establish whether it helped."

Then show the master deck's check-out slide (the Collective's check-out form, `bit.ly/oaisc-check-out`) and the stay-connected slide (Studio Hour).

## 7. Preparation checklist

- ☐ Host 1: confirm `python serve.py` and the 19 JavaScript tests on the actual host laptop. Time one baseline run and one benchmark run there.
- ☐ Host 1: keep a verified copy of the baseline and free-head `network.json` files outside `student/results/runs/`.
- ☐ Both hosts: make two **fresh** copies of the student kit from `packages/student-kit.zip`, one per chat, for the team prompts (6.5b). A rehearsal copy has edited files and extra runs; do not reuse it. Rehearse two or three invented team prompts and time them.
- ☐ Host 1: open ChatGPT Work in the **full** `puck-lab` **folder (the host kit), not the student kit,** for the real test. The student kit has no 96 held-out scenes or `host/score_submission.py`, so Improvement 3 cannot run there. In a test on the student kit, ChatGPT Work ran Improvements 1 and 2 correctly and then stopped at 3 and asked for the host scene file. Improvements 1 and 2 work in either folder.
- ☐ Host 1: rehearse the reference shot and the manual-push control in the browser, and confirm the presets and candidate slider match Section 3 (deterministic, seed 123). Record any difference.
- ☐ Host 1: rehearse `--data expanded` alone, since no result exists for it.
- ☐ Both: check the ChatGPT Work plan limits for a 60-minute session (up to two hours if you extend). The second host's account is the backup.
- ☐ Host 2: print 10 worksheets plus spares, pens, numbered hypothesis slides, team numbers, and a visible timer.
- ☐ Host 2: test projection legibility from the back of the room; enlarge the site and the terminal.
- ☐ Both: decide on a fresh benchmark set. If yes, generate it and freeze its hash now.
- ☐ Both: keep an offline copy of the whole folder. Hide notifications. Keep the demo tab in front while executing.
- ☐ Both: prepare labelled recordings of the reference shot and both repairs as fallbacks.

## 8. Failure plan

| Failure | Response |
|---|---|
| ChatGPT Work stalls for over 90 seconds | Discuss the prediction, then run the same command in a plain terminal and say so |
| Primary account unavailable | Switch to Host 2's laptop and account with the same folder |
| Network down | Everything runs locally except ChatGPT Work; run commands by hand and keep the votes going |

| Failure | Response |
| --- | --- |
| Playback looks stuck | Bring the demo tab to the front (it resumes); reload only if it stays locked, which clears the shot log, so keep screenshots |
| A run fails | Show the error, keep the folder, run the next voted option |
| Training is slower than planned | Use the saved checkpoints; label them as saved |
| Scoring cannot finish | Report the completed count only; make no full-result claim |

Never present saved output as the result of newly run code. Do not delete earlier runs to recover.

## 9. Printable team worksheet (one page)

```
PUCK LAB                                          Team: _______

Three questions: What does the model THINK will happen?
                 What ACTUALLY happens?  Where do they differ, and why?

The pipeline: state -> neural net -> predicted path -> chosen push
                                    the real simulator then runs the push

1. PREDICT (write before each reveal)
Shot 1: land in the ring?  [ ] yes  [ ] no    Shot 2: [ ] yes  [ ] no
What did the dashed line show, and what actually happened? _________

2. CHOOSE A FIRST CHECK
[ ] A. Collisions   [ ] B. Long free motion   [ ] C. The planner's picks
Our reason: ____________________________________________________
What did the host's check show? _____________________________________
What does it weaken or support? _____________________________________

3. PREDICT THE REPAIR (before each run)
Repair: _______  [ ] all fixed  [ ] some improve  [ ] no clear gain
Actual: ____ / 24   Mean miss: ______   New successes: ____   New failures: ____
What would change our mind? _________________________________________

4. YOUR TURN: our prompt for ChatGPT Work (ONE change)
Goal: ___________________________________________________________
Change: _________________________________________________________
Keep fixed: planner, scenes, seed.  We expect: ____________________
We would change our mind if: ____________________________________
Check: show hits, mean miss, new successes AND new failures.
While it runs, the stress test: [ ] same  [ ] different  [ ] unsure

5. JUDGE
[ ] Accept   [ ] Reject   [ ] Qualify
Benchmark hits: ____ / 96 vs ____ / 96   Regressions: ____
One result we trust: ____________________________________________
One uncertainty and next experiment: ____________________________
```

## 10. Activity slide and bench card

### Activity slide text

Title: **Activity**

> You'll direct ChatGPT Work to investigate a puck-planning model. Your team predicts what will happen, votes on what to try, and we run it live. Then you compare the results and decide: did the change actually help?

Shorter option: *Your team makes the calls. We run them live in ChatGPT Work. Predict, vote, compare the results, and decide whether the change really helped.* Use the same product name (ChatGPT Work, as in the deck) everywhere in the session. The hosts operate it, so do not promise that every attendee will use it themselves.

### When to walk the benches

Both hosts are busy during runs and votes, so circulate only in the gaps. With 10 teams and 2 hosts you cannot reach every team every round, so ask two or three KDAG organizers to float, one per three or four teams, and give each a copy of the card below.

| Moment | What the hosts do |
|---|---|
| Predict alone (1 min) | Host 2 stays at the front. Silence is fine. |
| Team discussion (1–2 min) | **Walk the benches.** This is the main window. |
| Your turn: writing the prompt (4 min) | **Walk the benches.** Ask: what do you expect, and what would change your mind? |
| Vote | Host 2 counts one raised hand per team and takes two 30-second reasons. |
| ChatGPT Work runs (15–90 s) | Host 1 narrates what ChatGPT Work is doing. Host 2 can keep circulating. |
| Result on screen | Both at the front. Read the result together. |

### Bench card (print one per floater)

```
BENCH CARD                                  Ask, do not tell.

ASK
1. What does your team think will happen, and why?
2. What result would change your mind?
3. Who disagrees? Give me the opposite argument.
4. What did the model PREDICT, what ACTUALLY happened, and where do they differ?
5. Did everyone write a prediction before the reveal?

WATCH FOR                       DO
Everyone agrees instantly       Ask one person to argue the other side
One person dominates            Swap their role with the recorder
No written prediction           Hold the reveal until they write one
Jargon confusion                Do not explain the maths. Point at the picture:
                                dashed line = what the model thinks,
                                solid line = what really happens
A team that is lost             Give a nudge question, not the answer

DO NOT
- Tell a team which repair works, or what the cause is.
- Touch their worksheet.
- Say a repair "works in general". Say "in this simulator, on these scenes".
```

### If you run several chats at once

If teams get their own ChatGPT Work chat instead of only the hosts operating it, set it up before the session:

- **One folder copy per chat.** ChatGPT Work edits files, and two chats on the same working folder will overwrite each other's runs and checkpoints.
- **One run-name prefix per team** (for example `t3-baseline`), so results never collide.
- **Check usage limits** for the whole session. Ten parallel sessions use far more than one projected session.
- **Keep the host workspace separate**, with the host model and final scenes. Do not copy it into a team chat.
- **Keep the same fixed rules.** The planner, scenes, seed and evaluator are not changed.

## 11. Claims discipline

- Say "in this fixed simulator, with one bumper, one seed, and these 96 scenes".
- Do not say a repair works "in general", is "robust", or is "significant".
- Report failures and regressions alongside gains.
- Do not credit one factor for the host model's 96/96: coordinates, architecture, seed and optimisation changed together.
- Hosts, not teams, run all code. Read any code before running it.

## 12. The improvement path in plain words (for newcomers)

Use this when someone asks "but what did you actually change?" or when the room is mostly beginners. Numbers marked *measured* come from our rehearsal on this machine (seed 42, 40 epochs) and must be re-checked on the day.

### The 30-second version (say this live)

> The model was bad at the rare, hard moments, and its small mistakes snowballed. So we gave the easy part its own simple helper, and then we checked whether it really worked.

### Words to avoid, and what to say instead

| Instead of | Say |
|---|---|
| neural network, model | "the puck predictor" (it guesses where the puck goes next) |
| one-step error | "how wrong it is on a single step" |
| rollout, recursive error | "building the path step by step, using its own guesses" |
| contact, collision response | "the moments the puck hits a wall or the bumper" |
| free motion | "gliding, when nothing is touching" |
| free-head, affine layer | "a small helper just for gliding" |
| validation, held-out | "scenes we did not tune on" |
| regression | "something that used to work and now doesn't" |

### Step by step

**Step 0: What the baseline is.**

- *Plain version:* it is a small neural network. You give it where the puck is and how fast it is moving (four numbers) and it guesses the same four numbers one step later. It learned this from about 24,000 "here's where the puck was, here's where it went" examples, with 40 passes over the data.
- *Show:* nothing yet, or the `run_experiment.py` command.
- *Evidence (measured):* it scores **2 of 24** development scenes, mean miss **0.385** table widths.
- *Do not say:* that the baseline is "broken". It is a reasonable starter that was trained on a narrow set of speeds.

**Step 1: See the problem.**

- *Plain version:* the dashed line (what the model thinks) says the puck lands right at the edge of the ring. The solid line (what really happens) misses.
- *Show:* the Lab, one shot, Execute. The dotted line between the endpoints is the gap.
- *Evidence (measured):* original network, Across, 512 candidates: predicted miss 6.8%, actual 29.1%.
- *Why it matters:* the model is confident and wrong. A prediction that looks fine is not evidence that it is right.

**Step 2: Ask where it goes wrong.**

- *Plain version:* there are two kinds of moments. The puck gliding on its own, and the puck hitting a wall or the bumper. The model is much worse at hits, and hits are rare in its examples.
- *Show:* the contact-vs-free table.
- *Evidence:* hits are about **2.9%** of the validation states and have about **11 times** the error (`host/results/diagnosis.json`).
- *The training data itself (counted 6 Oct 2026):* of the original 30,000 examples, **932 (3.1%) involve a hit**, and only **147 (0.5%) involve the bumper**. The puck speeds only go up to **0.040**, while the planner searches up to 0.11. So the model has seen hits, but rarely, and never at the speeds the planner uses. The expanded dataset has 23,336 hit examples (13.2%) and speeds up to 0.115.
- *Do not say:* that this *proves* hits are the cause. Say it *supports* the idea.

**Step 3: Try the obvious fixes and watch them fail.**

- *Plain version:* make hits count more, give it extra clues, or give it more data. These sound sensible.
- *Show:* the three runs against the baseline with `--compare baseline`.
- *Evidence (rehearsal record, seed 42, 40 epochs):*

| Try | Targets reached | Mean miss |
|---|---|---|
| Baseline | 2 of 24 | 0.385 |
| Count hits 5 times more (`--contact-weight 5`) | 0 of 24 | 0.545 |
| Extra clues about the walls and bumper (`--geometry`) | 0 of 24 | 0.410 |
| More data plus extra clues (`--data expanded --geometry`) | 0 of 24 | 0.508 |

- *Why it matters:* a sensible idea can make things worse. Keep the result and say what you expected.
- *If someone asks why they failed:* "we don't know yet." Possible reasons (more weight on hits can hurt gliding, which makes up most of the path) are guesses. Do not state one as fact.

**Step 4: Find the real clue.**

- *Plain version:* the path is 35 steps long. The model builds it step by step, using its own guess as the starting point for the next step. It is like copying a copy: tiny errors pile up until the path is far off.
- *Show:* the one-step vs path error plot (stretch block 1) if you have time.
- *Why it matters:* a model that is nearly right on every single step can still produce a badly wrong path.

**Step 5: The fix, `--free-head`.**

- *Plain version:* most of the path is gliding and slowly slowing down. That part is simple and steady, so we give it its own small learned piece, trained only on gliding examples. The neural network keeps the hard part, the hits.
- *What it really is (be open about this):* a tiny straight-line fit (a one-layer linear layer, fitted by least squares) on the non-contact examples only. At prediction time, any step that is not near a wall or the bumper goes to it. The code that decides "near a wall or the bumper" uses the table's known geometry. That is a prior we state openly, not something the network discovered. If someone says "that's just a linear fit", agree: it is, and it is learned from the data, not a physics formula.
- *Show:* the run command, then load the saved `network.json` through "Try your team's trained model" and execute.
- *Evidence (measured):* **13 of 24**, mean miss **0.101**, 11 scenes fixed, **no hits lost**. One scene (scene 10) is slightly farther than before (0.130 to 0.133 table widths). Say so; it is a good example of reporting what got worse.
- *Do not say:* that this is "the" solution, or that the room could have been expected to find it alone. Present it as a guided step.

**Step 6: Check it fairly.**

- *Plain version:* same scenes, same shots, same rules for both models. We score once on 96 scenes we did not tune on.
- *Evidence (measured, 5 Oct 2026):* **6 of 96 to 69 of 96**, mean miss **0.327 to 0.073**, 63 scenes fixed, 0 lost, 27 missed by both.
- *Show it honestly:* Diagonal is still a miss with the repaired model (about 22%). Say so out loud.
- *Bonus (measured):* the starter baseline predicts about a 3% miss on Across but actually misses by 55%. With the helper, prediction and reality agree (13.6% both), and more search finally helps.

**Step 7: The bigger idea (the reveal).**

- *Plain version:* describe each hit *relative to the surface it hit*. Then a hit at any angle looks the same to the model, so it learns one response and reuses it.
- *What the host model is:* free motion and walls use learned straight-line layers, the bumper uses a small learned network, and known geometry decides which one runs and how to turn the answer back into the table's coordinates.
- *Evidence (frozen set, one seed):* original 1 of 96, previous improved model 73 of 96, this model **96 of 96**.
- *Say plainly:* several things changed at once (coordinates, layer structure, seed, training setup), so we cannot say which one mattered most. It holds for one model in one simulator.
- *The legitimate route to 96/96, with no hand-given geometry:* (1) learn the gliding law and find the steps it cannot explain: those are collisions; (2) from the collisions, work out where the walls and the bumper are; (3) use those inferred positions to send each step to a gliding, wall or bumper piece; (4) learn each piece from data (the bumper in surface coordinates); (5) score once. We ran exactly this: **96/96** with the larger dataset and **94/96** with only the original 30,000 examples. The structure mattered more than the extra data. What is still assumed is the *form* of the table, not its numbers.

### Questions you will get

| Question | Answer |
|---|---|
| "Why didn't more data help?" | "Not on its own, in our test. With the extra clues it scored 0 of 24 at 40 passes. We haven't tried more data alone, and we don't know why." |
| "Is the helper really a neural network?" | "It's a tiny learned layer, a straight-line fit. The neural network still handles the hard moments in the bigger model. We're open that it's a structural idea." |
| "Isn't that cheating, since you gave it the geometry?" | "The geometry only decides which piece runs. The numbers inside each piece are learned from data. We state it as a prior. And we tested the opposite: a plain network with no geometry help, trained on the same large, fast dataset, scored 2 of 96 after 40 passes and 6 of 96 after 200 passes with a wider network, no better than the starter. So the structure, not just more data, seems to matter. That is one test, not a full ablation. We also went further: from the training data alone, with no constants given, we recovered the wall positions and the bumper's centre and radius to four decimals, and the whole model rebuilt on those inferred numbers scored 96 of 96 (94 of 96 using only the original 30,000 examples). What stays assumed is the form of the table (straight walls and a round bumper), not the numbers." |
| "Would it work on a different table?" | "We don't know. We tested one bumper, one simulator, one seed." |
| "Why not just use the physics equations?" | "That would work on this table, but it's a different experiment. Today we're testing whether a *learned* model can be trusted." |

## 13. Host script, slide by slide (60 minutes)

This follows the actual deck (19 slides, "ACTUAL WS PPT THNX") and the plan where the teams write their own prompts. Its slide timings replace any older clock times. **Arnav** operates the screen and the tool. **Ashish** facilitates, counts votes and keeps time (swap if you prefer). Words in quotes are spoken; [brackets] are actions. Numbers marked *measured* came from our rehearsal and must be re-checked on the day.

**Before the doors open:** student site on the projector, two ChatGPT Work chats open (one per host, each in its own copy of the student kit), a third window in the full `puck-lab` folder for the real test, the baseline and free-head `network.json` saved outside `student/results/runs/`, timer visible, and the slide 7 QR tested on a phone.

| Slides | Time | Segment |
|---|---|---|
| 1–2 | 0:00–0:05 | Check-in and title |
| 3–4 | 0:05–0:07 | The Collective and campus leads |
| 5 | 0:07–0:10 | Form groups |
| 6–8 | 0:10–0:15 | ChatGPT Work, activity, activity title |
| 9–10 | 0:15–0:21 | The puzzle and what to try |
| 11–13 | 0:21–0:28 | Choose an idea, starter prompt, discuss |
| 14 | 0:28–0:36 | Two worked examples |
| 14 | 0:36–0:42 | Your turn: teams write a prompt |
| 14 | 0:42–0:52 | Hosts run two team prompts, one stress case |

| Slides | Time | Segment |
|---|---|---|
| 15 | 0:52–0:57 | Real test, vote, spotlight |
| 16–19 | 0:57–1:00 | Check-out, stay connected, close |

### Slides 1–8 (0:00–0:15)

Unchanged: see below.

**Slide 1, check-in (0:00–0:03). Ashish:** "Hi everyone, welcome! Scan the code on screen and check in. It takes ten seconds. Then grab a seat with the people around you." **Arnav:** [Opens the site and the tool. Checks both load.]

**Slide 2, Call Your Shot! (0:03–0:05). Ashish:** "Today's session is called **Call Your Shot**. Think, Build, Benchmark. By the end of the hour you'll have predicted what an AI model does, written your own prompt to fix it, and checked whether the fix really worked."

**Slide 3, the Collective (0:05–0:06). Ashish:** "The OpenAI Student Collective brings students together to try the newest AI tools, through workshops, challenges and campus events."

**Slide 4, campus leads (0:06–0:07). Arnav:** "I'm Arnav, third year, chemistry and financial engineering. My favourite fun use of ChatGPT is decoding random internet lore." **Ashish:** "I'm Ashish, third year, chemistry. Mine is reading my notes for me."

**Slide 5, form a group (0:07–0:10). Ashish:** "Get into groups of six to seven. No pun intended. Introduce yourselves: name, major, year, hometown. Then tell the group something you've been unusually into lately. You're a team for the whole hour. Pick a **discussion lead**, a **recorder** and a **spokesperson**. The spokesperson casts your vote and, later, hands in your prompt." [Walk around for 90 seconds.]

**Slide 6, ChatGPT Work (0:10–0:12). Arnav:** "ChatGPT Work is built for bigger jobs. Give it a plan and it can take on a whole project. It connects to your tools and browser, and it can run on a schedule. Today we give it a bounded task, then check what it did."

**Slide 7, activity (0:12–0:14). Ashish:** "You'll direct ChatGPT Work to investigate a puck-planning model. Your team predicts, votes, and writes a prompt. We run it live. Scan this code to open the site; no login needed." **Arnav:** "We run the tool on the big screen, so nobody is blocked. You make the calls."

**Slide 8, Puck Around, Find Out! (0:14–0:15). Arnav:** "This activity is called **Puck Around, Find Out**. Try things with AI. Keep what holds up."

### Slide 9: The puzzle (0:15–0:20)

**Arnav:** "I used ChatGPT Work to plan a puck's path to a target in 35 steps, using a small neural network. Here's the problem." [Original neural network, Across, 512 candidates. Find a new shot.] **Ashish:** "Will the puck land in the green ring? Write yes or no." [30 seconds.] "Hands up for yes… and no." **Arnav:** [Execute.] "Predicted miss about 6.8%. Actual miss about 29.1%. *Measured.*" **Ashish:** "Same push. The model predicted it would land right at the edge of the ring, and reality disagreed." **Ashish:** "A neural network is a small guesser. It sees where the puck is and how fast it's moving, and guesses where it will be one step later. Do that 35 times and you get the dashed path. No idea? Ask GPT for a few lines, or ask your neighbours." **Ashish:** "If you have a laptop, the Team kit tab has the files for later. Today we run everything live, so you don't need to install anything."

### Slide 10: What you all should try (0:20–0:21)

**Ashish:** "Three things you can do with a model like this. **Diagnose**: find where prediction and reality split apart. **Repair**: change how the network is trained and compare. **Stress-test**: move the target, change the speed, add more search."

### Slides 11–13: Diagnose (0:21–0:28)

**Slide 11 (0:21–0:23).** [Paste the starter prompt into chat A now so the answer is ready by slide 12: it took about a minute in testing.] **Ashish:** "One minute in your team. What will the model get wrong first: collisions, long free motion, or the planner's picks? One hand per team." [Count. Two 30-second reasons.] **Slide 12 (0:23–0:26). Arnav:** [Shows the answer.] "Outcome, context, constraints, and a check. The constraint was: don't edit any files. It found the function that picks the push and the one that predicts paths." [Contact-vs-free table.] "Collisions are about 3% of the training data, but the error there is about eleven times bigger. *Measured.*" **Ashish:** "Which of your votes does that support? It supports it. It doesn't prove it." **Slide 13 (0:26–0:28). Ashish:** "Ninety seconds. What did the model predict, and what actually happened? Where do the paths differ? What one thing would you try, and what result would change your mind?" [Walk the benches. Model answer in Section 6.]

### Slide 14: Take your idea further (0:28–0:52)

**Worked example 1 (0:28–0:32). Arnav:** [Runs the baseline.] **Ashish:** "All fixed, some better, or no change? Tick your sheet." **Arnav:** "The starter scores 2 out of 24, mean miss 0.385. *Measured.*" [Runs collision weight 5x.] "Zero out of 24, mean miss 0.545. Worse." **Ashish:** "A sensible idea that made it worse. We keep that result, and we don't yet know why."

**Worked example 2 (0:32–0:36). Ashish:** "Here's a different idea. Most of the path is gliding, which is simple and steady. So we give the gliding its own small learned helper." **Arnav:** [Runs the free-motion head.] "13 out of 24, mean miss 0.101, and no hit was lost. One scene's miss grew slightly, so we say that too. *Measured.*" [Loads the baseline weights, executes, then the free-head weights.] "The starter predicted about a 3% miss and actually missed by 55. Now it predicts 13.6% and misses by 13.6%. It's honest about what it can do."

**Your turn: write (0:36–0:40). Ashish:** "Improvement 3 is yours: your idea. Use your own brains. Pick ONE change. Say what you expect, and what result would change your mind. Write it as a prompt for ChatGPT Work: a **goal**, one **change**, what you **keep fixed**, and a **check**. Stuck? Try a wider network, more training passes, or extra inputs about the walls and the bumper. Or ask GPT for ideas. Four minutes." [Both hosts walk the benches: "What do you expect? What would change your mind?"]

**Hand in (0:40–0:42). Ashish:** "Spokespeople, read your prompt." [Pick two different ideas. For any other: "Great idea, not today. We'll note it."] **Arnav / Ashish:** [Add the guardrail suffix, paste one prompt each into chat A and chat B.]

**Hosts run team prompts (0:42–0:52). Ashish:** "While these run, let's break something." [Stress case: Diagonal, then 2,048 candidates if there is time.] **Arnav:** "With the repaired model, Diagonal is still a miss, about 22%. That's a limitation and we say it out loud. More search lands Across, about 1.5%, because the predictions are honest now. *Measured.* These are chosen stress cases, not a robustness benchmark." At 0:48 read each result: **Ashish:** "Team 4 asked for a wider network. Here's what happened: hits, mean miss, new successes, and what got worse." [Credit the team. If a chat is still working at 0:50: "We'll check back at the end."]

### Slide 15: Highlights (0:52–0:57)

**Ashish:** "Now the real test. You chose your ideas using the 24 development scenes, so those scores are a bit optimistic. We score once on 96 scenes nobody tuned on. Freeze, no more edits." **Arnav:** [In the full folder: runs the baseline and the best candidate.] "Baseline 6 out of 96, mean miss 0.327. Free-motion candidate 69 out of 96, mean miss 0.073. 63 scenes fixed, no hits lost, 27 still missed. *Measured.*" **Ashish:** "It helped on these scenes. It's one model, one seed, one simulator. We won't say robust." **Ashish:** "Vote: accept, reject or qualify." [Hands. Two teams, 30 seconds each, citing something they saw.] **Arnav:** [Skip the rest if short on time.] [Host view.] "Our own model scored 96 out of 96 on the same scenes. It treats gliding and hitting as two different problems. For hits, it describes each collision relative to the surface it hit, so a hit at any angle shares one learned response." **Ashish:** "Fair question: did we cheat by telling it where the walls and the bumper are? So we tested it. We threw away our hand-written numbers and let the data tell us. From the training data alone, it finds the walls and the bumper to four decimal

places. The same model, built on those found numbers, still scored 96 out of 96. And using only the original small dataset it scored 94 out of 96. *Measured.* Show the prepared numbers; do not run it live." **Arnav:** "What we still assume is the shape of the table: straight walls and one round bumper. The numbers are learned. We changed several things at once, so we can't say which one mattered most, and it's one model in one simulator. But the lesson holds: look at where the model goes wrong, give that part the right structure, and test it fairly."

### Slides 16–19 (0:57–1:00)

**Slide 16. Ashish:** "Scan the code to share what you made with the OpenAI team and get a custom graphic. Three takeaways: confident isn't the same as correct, a sensible fix can fail so keep the result, and compare results including what got worse. ChatGPT Work can implement the change. Your job is to establish whether it helped." **Slides 17–18. Arnav:** "Join the campus platform, follow our Luma page for events, and subscribe on Substack. Studio Hour details will be shared soon." **Slide 19. Ashish:** "Fun fact: all the elements in these slides were generated by GPT. Hope it looked good. Thanks, everyone!"

## 14. Commands: score and build both approaches

**See every score in one table** (about 1 minute for five models on both scene sets, then delete `scratch/`). The first model is the baseline; `--dev` adds the 24 development scenes:

```
python host/geometry_from_data_pipeline.py scratch/geofree expanded
python host/geometry_from_data_pipeline.py scratch/geofree-orig original
python host/show_scores.py scratch/baseline.json scratch/candidate.json host/local_neural_repair.py
scratch/geofree/repair_local_inferred.py scratch/geofree-orig/repair_local_inferred.py --dev
```

Measured 6 Oct 2026 (`scratch/baseline.json` and `scratch/candidate.json` are the starter baseline and the free-motion run copied from your chat copy's `student/results/runs/`):

| Model | 96 hits | 96 mean miss | 24 hits | 24 mean miss | Fixed vs baseline | Lost |
|---|---|---|---|---|---|---|
| Starter baseline | 6/96 | 0.327 | 2/24 | 0.385 | (base) | |
| Free-motion helper | 69/96 | 0.073 | 13/24 | 0.101 | 63 | 0 |
| Known-geometry model | 96/96 | 0.011 | 24/24 | 0.010 | 90 | 0 |
| New approach, larger data | 96/96 | 0.013 | 24/24 | 0.012 | 90 | 0 |
| New approach, original data only | 94/96 | 0.014 | 24/24 | 0.011 | 88 | 0 |

Or score one model at a time:

Run everything from the full `puck-lab` folder (the host kit). Scoring writes its report next to the file it scores, so score copies in `scratch/`, and delete `scratch/` afterwards.

**Score the known-geometry model** (about 20 seconds, expect `confirmation: 96/96 targets.`):

```
python host/score_submission.py host/local_neural_repair.py
```

**Build and score the new approach, larger dataset** (about 35 seconds to train, then 20 to score; expect `96/96`):

```
python host/geometry_from_data_pipeline.py scratch/geofree expanded
python host/score_submission.py scratch/geofree/repair_local_inferred.py
```

**New approach, strict version using only the original 30,000 transitions** (about 5 seconds to train; expect `94/96`):

```
python host/geometry_from_data_pipeline.py scratch/geofree-orig original
python host/score_submission.py scratch/geofree-orig/repair_local_inferred.py
```

**Build the two sites:**

```
python scripts/build_student_site.py     (writes dist-student/: no host models, no results page)
python scripts/build_host_site.py        (writes dist-host/: both models, results, pointers; keep it
private)
```

**Run them locally:** `start.ps1` serves the host site at http://127.0.0.1:8766/, and `start-student.ps1` serves the student site at http://127.0.0.1:8767/.

Notes: the new approach is also selectable in the Lab in host view ("New approach: geometry inferred from data"). Score once, do not tune afterwards, and call the 96-scene set a public benchmark.